

IntentFlow

AI SEO Intelligence Platform

Captures invisible user behaviour from AI chat providers
and materialises it into structured, actionable campaign data.

3

SURFACES

4+

AI PROVIDERS

15

DB ENTITIES

NODE.JS

REACT 19

CHROME MV3

POSTGRESQL

BULLMQ

PRISMA

TYPESCRIPT

Sibtain · Abhishek · Amogha · Yaseen · Saksham · Saumya

Table of Contents

01	Executive Summary
02	Technical Architecture
03	System Design Principles
04	Object-Oriented Design
05	Database Design
06	Data Flow
07	Team — Roles & Contributions
08	Future Scope & Roadmap

IntentFlow is a full-stack AI SEO intelligence platform. This document covers architecture, design decisions, database schema, data flow, and team contributions.

Bridging the Visibility Gap in AI-Driven Search

Traditional SEO tools monitor Google searches. As users migrate to conversational AI — ChatGPT, Claude, Perplexity, Grok — those queries and the sites AI models cite are entirely invisible to standard analytics. IntentFlow bridges this gap by intercepting live AI response streams, parsing them into hierarchical campaign trees, and enriching them with third-party SEO data from SEMrush and Ahrefs — all in real time.

Every AI conversation a user has becomes a structured, deduplicated, version-controlled SEO intelligence record that operators can explore, compare over time, and act on.

The Core Problem

Challenge	Impact
Millions of searches now happen inside AI chatbots	Zero signal in Google Analytics or SEMrush
AI-recommended websites are not tracked	Missed SEO opportunities and competitor intelligence
No standard tool intercepts conversational AI intent	Marketers operate blind in the fastest-growing search channel

Core Features

Feature	Description
Multi-Provider Interception	Captures live streams from ChatGPT, Claude, Perplexity, and Grok simultaneously via a Chrome Mar
Deterministic Tree Mapping	Every conversation materialises into a stable Prompt → Subquery → Site tree using deterministic SH
Campaign Versioning	Snapshot campaigns and refire them to track how AI-cited sites change over time.
SEO Enrichment	Background queues (BullMQ) auto-enrich sites with SEMrush and Ahrefs keyword volume, traffic esti
Lead Intelligence	Extracts buying-intent signals from raw prompt text and scores users by behavioural intent.
Admin RBAC Dashboard	Granular, multi-tenant role-based access control with full administrative visibility.
AI Prompt Suggestions	Uses OpenAI GPT to suggest follow-up prompts based on campaign history.

Three Independently Deployable Surfaces

IntentFlow is a monorepo with three surfaces communicating over HTTPS via REST APIs and JWT authentication, backed by a shared PostgreSQL + Redis data layer.

Surface	Responsibility	Key Technologies
Chrome Extension (MV3)	Injects MAIN-world content scripts into AI provider pages to intercept SSE streams, GraphQLs, and SPAs. Side-panel UI for campaign exploration.	Chrome MV3 · GraphQL · SPAs React · Vite
Backend API (Node.js)	Ingests raw events, materialises campaign trees using deterministic SHA-256 hashing, enriches with BullMQ on Redis.	Express.js · Prisma · BullMQ · JWT · Zod
Web Dashboard (React 19)	Campaign graph exploration (XYFlow), analytics (Recharts), admin controls (React 19), and lead intelligence (OpenAI GPT).	React 19 · XYFlow · Recharts Tailwind

Technology Stack

Layer	Technology
Backend Runtime	Node.js 20+ · TypeScript
Web Framework	Express.js 4
Database	PostgreSQL via Prisma ORM
Queue / Jobs	BullMQ on Redis
Authentication	JWT access tokens + DB-backed refresh tokens
Frontend	React 19, Vite, Tailwind CSS, Shadcn UI / Radix UI
Visualisation	XYFlow (campaign trees) · Recharts (analytics)
Extension Build	Chrome MV3 · CRXJS plugin · Vite
Validation	Zod (DTO schema validation)
External APIs	OpenAI GPT · SEMrush · Ahrefs

Engineering Decisions That Scale

3.1 Layered Architecture (N-Tier)

Strict 4-tier separation: Routes → Controller → Service → Repository. Swapping PostgreSQL for another database only touches the Repository layer. Each layer only speaks to the one directly below it.

```
Routes → Controller → Service → Repository
if (exists) return existing; else create and return new;
```

3.2 Multi-Tenancy

Every database record carries a `tenant_id`. Row-level isolation is enforced in the Repository layer — no tenant can ever access another's data, even on the same server.

```
WHERE tenant_id = ? on every query
Enforced in Repository, not business logic
```

3.3 Event-Driven / Queue-Based

Heavy enrichment tasks (SEMrush, Ahrefs, NLP) are pushed to BullMQ queues, never blocking the HTTP response. Producer-Consumer pattern throughout.

```
ingestTurn() → push job → return 201
Worker processes async in background
```

3.4 Idempotency

`ingestTurn()` checks `request_id` + `turn_exchange_id` before writing. Duplicate events from network retries return the existing tree instead of creating duplicates.

```
if (exists) return existing;
else create and return new;
```

3.5 Deterministic Hashing

Internal IDs use SHA-1 hashes of content rather than random UUIDs. The same prompt from the same provider always produces the same hash — enabling reliable deduplication.

```
SHA1(provider + turn_id + query + site)
→ always same result_ref for same content
```

3.6 Caching + RBAC

SEMrush/Ahrefs results cached in `SemrushSnapshot` with 24-hour TTL. Dual-role RBAC (App Role: admin/user · Tenant Role: owner/member) enforced via JWT middleware on every route.

```
authMiddleware → requireRole('admin')
→ requireAccountRole('owner')
```

Full OOP in TypeScript

IntentFlow uses TypeScript's complete OOP feature set — not just as a style choice, but because each pattern directly solves a specific architectural problem.

Pattern	Implementation	Code Example
Abstraction Abstract Base Classes	Three abstract classes — BaseController, BaseService, BaseRepository — CampaignService contracts for the entire ba	<pre>library - CampaignService extends BaseService</pre>
Encapsulation Private Methods	CampaignService hides resolveVersion(), toVersionSummary(), toValueSummary() as private methods. Public A	<pre>private listExecutedPrompts()</pre>
Inheritance Class Hierarchy	Concrete classes extend abstract bases, inheriting shared utility methods. Adding utility to BaseService automatically	<pre>CampaignController extends BaseController AuthRepository extends BaseRepository</pre>
Polymorphism Generics + Runtime	ApiResponse.success returns different shapes per type T. StreamApiResponse.successes<Campaigns>(data) same int	<pre>ApiResponse.successes<Campaigns>(data) ApiResponse.success<User[]>(data)</pre>
Singleton Single Instances	Services, controllers, and repositories are instantiated once at module load. Prevents multiple DB connections and e	<pre>new CampaignService(repo)</pre>
DI + Factory	CampaignService receives its repository via constructor injection — consistent (testing with mock repos. ApiResponse	<pre>constructor(private repo: CampaignRepository)</pre>
Strategy Pattern Per-Provider	Each AI provider (ChatGPT, Claude, Perplexity, Grok) has its own content stream implementing the same event-posting	<pre>claudeStreamContent.ts perplexityStreamContent.ts grokStreamContent.ts</pre>

15 Entities Around a Self-Referencing Tree

PostgreSQL managed via Prisma ORM. The central data model is a PromptNode adjacency list — a single table representing an unbounded Prompt → Subquery → Site hierarchy.

Entity	Purpose
User	Platform user — email, password hash, app role, lead score data.
Tenant	Organisation/account boundary. All data is scoped to a tenant.
TenantMember	Join table: User and Tenant with role (owner / member).
Campaign	An SEO campaign linked to a Tenant and optionally a Domain.
CampaignVersion	Point-in-time snapshot of a campaign. Can be archived and refired.
CaptureSession	One AI chat conversation linked to a CampaignVersion.
CaptureTurn	One prompt-response exchange within a session. Stores raw_event_json.
PromptNode ★	Self-referencing tree node. Types: prompt, subquery, site, generated.
SemrushSnapshot	Cached keyword and traffic data from SEMrush / Ahrefs. TTL: 24h.
LeadSignal	Buying-intent signal extracted from a capture turn.
Domain	A tracked website domain with scrape status and context JSON.
GenerationRun	Tracks an AI suggestion generation job (queued/processing/done).
AppRole	Enum: admin · user
TenantRole	Enum: owner · member
AiChatProvider	Enum: chatgpt · claude · gemini · perplexity · grok · unknown

The Campaign Tree — Adjacency List

```
// PromptNode – self-referencing, parent_id points to another PromptNode
● prompt 'best SEO tools 2026' parent_id = null (root)
■ subquery 'top seo software' parent_id = prompt.id
■ site ahrefs.com parent_id = subquery.id
■ site semrush.com parent_id = subquery.id
■ generated 'compare seo pricing' source = 'suggestion'

// All IDs are deterministic SHA-1 hashes – not random UUIDs
```


Extension → Backend → Dashboard

The full end-to-end pipeline from a user typing a prompt in ChatGPT to a fully enriched campaign tree visible in the web dashboard.

1 User opens ChatGPT / Claude / Perplexity

The extension's MAIN-world script is injected by Chrome via manifest.json — bypassing Content Security Policy. It patches the browser's native fetch/XHR.

```
chatgptStreamMain.ts · world: MAIN
```

2 Live stream intercepted and parsed

The MAIN-world script intercepts SSE/streaming responses. Extracts: the user's prompt, any internal search queries the AI ran, and websites cited in the response.

```
streamMain.ts → window.postMessage → streamContent.ts
```

3 Data relayed to background service worker

The isolated-world content script relays parsed events to the background worker via chrome.runtime ports. Background batches events and calls the backend API.

```
chrome.runtime.connect() → background.ts
```

4 POST /campaigns/:id/ingest-turn

Structured JSON payload sent to backend with JWT auth. Zod validates the DTO. Controller delegates to CampaignService.ingestTurn().

```
authMiddleware → CampaignController → CampaignService
```

5 Campaign tree materialised in PostgreSQL

Service normalises data, resolves the active CampaignVersion, computes deterministic SHA-1 hashes, and upserts PromptNode records via the Repository.

```
CampaignService → CampaignRepository → Prisma → PostgreSQL
```

6 Enrichment jobs queued asynchronously

SEMrush and Ahrefs jobs are pushed to BullMQ — the HTTP response returns immediately. Workers process in parallel, store SemrushSnapshot records with 24-hour TTL.

```
BullMQ → semrush.worker.ts → SemrushClient → API
```

7 Lead intelligence extracted (fire-and-forget)

LeadIntelligenceService.extractSignalsFromTurn() runs asynchronously — scoring buying intent, product interest, and behavioural signals from the raw prompt text.

```
LeadIntelligenceService → LeadSignal table
```

8

Dashboard renders the live campaign tree

Extension side-panel and web dashboard fetch the enriched tree via GET /campaign/:id/active-tree. XYFlow renders the interactive graph; Recharts shows analytics.

React Dashboard → XYFlow graph · Recharts analytics

Built by Six

IntentFlow was designed and built by a six-person team, each owning a distinct engineering domain. Below is a detailed breakdown of each member's role and key contributions.



Sibtain Ahmed Qureshi

PROJECT LEAD & ARCHITECT

◆ Node.js ◆ Prisma ORM ◆ PostgreSQL ◆ TypeScript ◆ Express.js

- Designed the overall monorepo structure and surface interaction model
- Built the deterministic materialization engine (SHA-1 tree mapping)
- Architected the Prisma schema and PostgreSQL migration strategy
- Defined the 4-tier layered architecture (Routes → Controller → Service → Repository)
- Established the campaign versioning and refiring system
- Integrated multi-tenancy model with tenant_id row-level security



Abhishek Verma

STRATEGY & SYSTEMS DESIGN

◆ RBAC ◆ JWT ◆ System Design ◆ Zod ◆ Security Architecture

- Conceptualised the lead intelligence framework and intent scoring models
- Established Agent Context governance for AI-assisted development
- Designed multi-tenant RBAC security architecture (App Role + Tenant Role)
- Defined the dual-role JWT middleware pipeline and authorization flow
- Drove overall system governance and technical decision documentation
- Mapped subsystem boundaries and module interface contracts



Amogha Raj Sandur

DEVOPS & QUALITY ASSURANCE

◆ CI/CD ◆ pnpm ◆ DevOps ◆ QA ◆ Documentation

- Managed CI/CD alignment and task-to-milestone mapping
- Led monorepo refactoring for modularity and low technical debt
- Architected handoff documentation and progress tracking standards
- Established code quality standards and review processes
- Configured linting and build tooling across all three surfaces
- Ensured consistent pnpm workspace setup and dependency management



Mohammed Yaseen

FULL-STACK ENGINEERING

◆ React 19 ◆ Chrome MV3 ◆ Vite ◆ TypeScript ◆ REST APIs

- Built mock server and testing ecosystem for provider-free local development
- Led cross-surface sync between extension side-panel and web dashboard
- Implemented onboarding and analytics flows end-to-end
- Built web bridge for extension ↔ localhost development sync
- Contributed to Chrome MV3 service worker orchestration
- Developed the extension's side-panel React UI and hash-router setup



Saksham

BACKEND & DATA PIPELINES

◆ BullMQ ◆ Redis ◆ SEMrush API ◆ Ahrefs API ◆ NLP ◆ Node.js

- Built SEMrush and Ahrefs API bridge clients and normalisation layers
- Optimised BullMQ queue configuration for concurrent enrichment throughput
- Implemented NLP intent extraction pipeline for lead scoring
- Designed SemrushSnapshot caching model with 24-hour TTL
- Built LeadIntelligenceService and LeadSignal persistence
- Contributed to background worker architecture and retry strategies



Saumya Mishra

FRONTEND & UX ENGINEERING

◆ React 19 ◆ XYFlow ◆ Recharts ◆ Tailwind CSS ◆ Shadcn UI ◆ Figma

- Integrated XYFlow for interactive campaign tree graph visualisation
- Built Compact Tree Explorer with zoom, pan, and node detail views
- Maintained and extended the design system (Shadcn UI + Tailwind tokens)
- Designed and implemented the web dashboard layout and navigation
- Built Recharts analytics dashboards for prompt and website data
- Led responsive UI polish and component library consistency

What Comes Next

NEAR-TERM

Server-Driven Replay and Automation

Move campaign refiring from the extension to a fully server-side model — enabling scheduled background re-runs and monitoring without requiring an active browser session.

MID-TERM

Advanced Lead Scoring and CRM Integration

ML models refined on historical interaction data. Automated export pipelines to Salesforce and HubSpot via custom webhooks.

MID-TERM

Enterprise Observability and Scaling

OpenTelemetry integration for end-to-end event tracing. Scaling the Redis job queue to handle thousands of concurrent enterprise ingestion streams.

LONG-TERM

Pilot Readiness and Commercial Launch

Multi-tenant billing infrastructure, SOC2 compliance readiness, and high-availability deployment hardening for public pilot launch.
